

Počítačové a komunikačné siete

Komunikácia s využitím UDP protokolu

Meno: Bálint Janik

Ais ID: 127174

Šk. Rok: 2024/2025

Cvičenie: Ing. Adrián Ondov, Ut 18:00

Obsah dokumentu

Cieľ zadania

.....	1
-------	---

Analýza zadania

Aplikačná vrstva.....	1
Transporná vrstva.....	1
UDP.....	1

Návrh riešenia

Programovací jazyk a používateľské rozhranie.....	4
Spojenie a komunikácia.....	4

Hlavička protokolu

Komunikácia.....	5
Prenos dát.....	5-6
Overenie integrity.....	6

Diagram komunikácie

.....	7
-------	---

Funkcionality program

Inicializácia komunikácie.....	8
ARQ metóda.....	9
CRC16 Checksum.....	9
Keep-Alive metóda.....	9

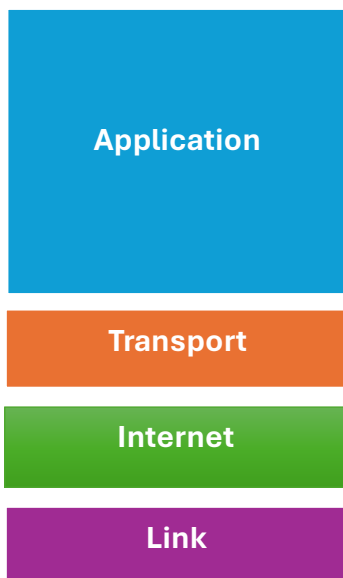
Cieľ zadania

Navrhnuť a implementovať P2P aplikáciu využívajúcu vlastný protokol postavený nad UDP (User Datagram Protocol) v transportnej vrstve sieťového modelu TCP/IP. Aplikácia bude umožňovať komunikáciu dvoch účastníkov v lokálnej Ethernet sieti, vrátane výmeny textu a prenosu ľubovoľných súborov medzi počítačmi (uzlami). Oba uzly budú fungovať súčasne ako prijímač aj odosielateľ.

Analýza zadania

V rámci sieťovej komunikácie môžeme hovoriť o nespočetne veľa protokoloch, ktoré musia spolupracovať pre zabezpečenie správneho prenosu dát. Pri zadaní budeme pracovať s modelom TCP/IP.

TCP/IP



Aplikačná vrstva

Aplikačná vrstva je najvyššia vrstva. V referenčnom modeli je rozdelená na Application, Presentation a Session, ale pri TCP/IP modeli hovoríme o jednej vrstve, ktorá slúži na komunikáciu medzi koncovými aplikáciami. **Na tejto vrstve sa bude nachádzať aj naša hlavička.**

Transportná vrstva

Táto vrstva zabezpečuje komunikáciu medzi rozličnými hostami, ktorí sú adresovaní jednotlivými portami. Patria sem známe protokoly ako TCP, DCCP, ale aj **UDP**.

UDP (user datagram protocol)

UDP je nespojovo orientovaný protokol. Síce umožňuje rýchly prenos dát, keďže je ľahký, bezstavový, ale je nespoľahlivý, keďže nerieši problém strát. Každý paket sa posiela nezávisle, pri strate packet nie je automaticky znovu odoslaný. Používa sa pri aplikáciách, kde je dôležitá rýchlosť prenosu a nízka latencia ako napr. Online hry alebo streamovanie.

Návrh riešenia

Programovací jazyk a používateľské rozhranie

Protokol budeme implementovať v jazyku Python a to hneď z niekoľkých zásadných dôvodov. Python má veľké množstvo funkcií vďaka ktorým obídeme potrebu manuálnej implementácie niektorých funkcií a s počtom knižníc, ktoré má python k dispozícii je tento jazyk praktickou voľbou. Na ovládanie našej aplikácie budeme využívať CLI, ktorý upravíme prehľadne do zrozumiteľnej a ľahko operatívnej podoby. Testovanie bude prebiehať na zariadeniach s OS MacOS.

Spojenie a komunikácia

Keďže ide o P2P komunikáciu nebudeme určovať rolu „servera“ a „clienta“, keďže v našej aplikácii budú komunikáciu vykonávať iba „clienti“, ktorý sa na seba na začiatku pripoja a inicializujú komunikáciu na základe porovnania IP adries.

Inicializácia spojenia: Klient s nižšou IP adresou pošle prvotný handshake paket obsahujúci detaily o pripojení. Druhý klient odpovie potvrdzujúcim ACK packetom, na ktorý prvý klient rovnako odpovie ACK packetom. Výkonaním tohto handshake-u je inicializácia úspešná.

Odosielanie a prijímanie dát: Po úspešnom nadviazaní spojenia sa klienti môžu rozhodnúť, či chcú posilať správy alebo súbory. Keďže ide o P2P, každý klient môže iniciovať posielanie dát. Používame vlastný protokol, ktorý podporuje fragmentáciu paketov – fragmenty sú odosielané jeden po druhom, pričom klient čaká na potvrdenie prijatia každého fragmentu.

Fragmentácia a znovuzaslanie: Ak správa alebo súbor presiahne maximálnu veľkosť paketu, dáta sú rozdelené do fragmentov. V každom fragmente je uvedené sekvenčné číslo (na identifikáciu poradia správ) a fragment číslo (na identifikáciu jednotlivých častí jedného celku). Ak nedôjde k prijatiu potvrdenia do určitého časového limitu, fragment sa opätovne odošle. Klienti si môžu nastaviť veľkosť fragmentu.

Kontrola integrity dát: Každý fragment obsahuje CRC16 kontrolný súčet, ktorý zabezpečí, že fragmenty budú overené pri prijatí a akékoľvek poškodené fragmenty budú znovu vyžiadané od odosielateľa.

Záverečná synchronizácia: Po odoslaní všetkých fragmentov sa klienti synchronizujú a odosielajú finálne potvrdenie, ktoré potvrdí kompletnosť prenosu.

Hlavička protokolu

Komunikácia

Najprv si určíme kľúčové časti zabezpečujúce samotný handshake mechanizmus a základy komunikácie:

Pole	Veľkosť	Funkcia
Version	4 bity	Základná identifikácia packetu
Flags	12 bitov	Rôzne príkazy pre funkcie

Prvá identifikovaná časť je vždy Version, ktoré obsahuje rovno niekoľko kľúčových bitov. Na základe prvého bitu vieme určiť kto inicializoval komunikáciu, čo nám umožní packety rýchlejšie spracovávať. Dva bity máme vzhradené pre špeciálne prípady a to pre ACK a FIN packety. A posledný bit máme rezervovaný pre budúce úpravy.

Flags využívame na identifikáciu každého jedného packetu komunikácie, nachádzajú sa tam príkazy aj pre handshake, ale môžu byť doimplementované príkazy ako PUSH alebo URGENT na spávu metódy posielania dát alebo ERROR na riešenie chzbných stavov. Bity 5 – 12 sú rezervované na budúce úpravy.

Prenos dát

UDP má obmedzenie na veľkosť packetu (zvyčajne okolo 65 535 bajtov vrátane hlavičky), takže musíme implementovať mechanizmus fragmentácie pre väčšie správy a súbory.

Pole	Veľkosť	Funkcia
Version	4 bity	Základná identifikácia packetu
Flags	12 bitov	Rôzne príkazy pre funkcie
Sequence Number	16 bitov	Číslo sekvencie
Ack Number	16 bitov	Číslo potvrdenia
Payload Length	16 bitov	Dĺžka dátovej časti
Fragment Number	8 bitov	Číslo fragmentu
Total Fragment	8 bitov	Celkový počet fragmentov

Keďže dáta prenášané pomocou nášho protokolu môžu byť fragmentované a protokol UDP je nespoľahlivý musíme riešiť rovno niekoľko problémov.

Dáta môžeme mať rozfragmentované, ale zároveň môžeme poselať viacero dát naraz, preto potrebujeme vedieť čo kam patrí. Na to využijeme sequence number a acknowledgement number, kde tieto čísla označujú ktoré dáta v poradí sa posielajú

a druhá strana pre bezpečnostné dôvody potvrdzuje, že tieto dáta dostala. Je to výborný spôsob ako riešiť stavy, kedy sa dáta neposielajú v poradí.

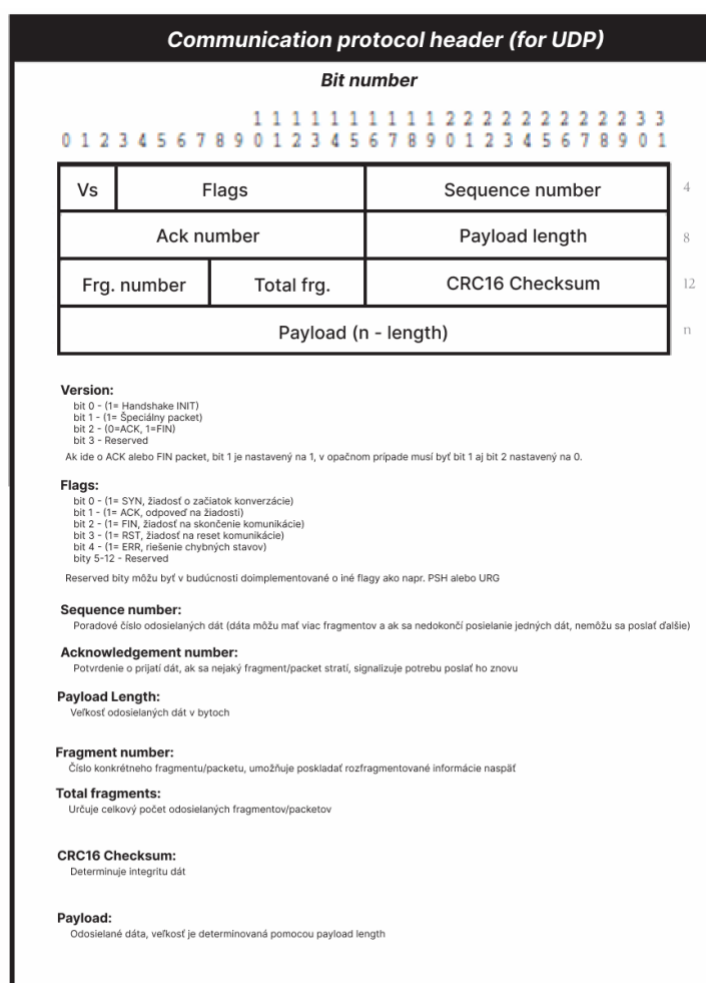
Tým, že sa tie dáta neposielajú v poradí a môžu byť rozkúskované na menšie dáta, teda fragmentované, musíme si pamätať aj poradie fragmentov aby dáta opačná strana vedela naspäť poskladať. Samozrejme poskladať ich bude vedieť iba v tom prípade ako obdržala rovnaký počet fragmentov ako je uložené v poli total fragments.

Overenie integrity a ARQ

Pre kontrolu integrity použijeme checksum – presnejšie CRC16 (Cyclic Redundancy Check). Ide o bežný mechanizmus na detekciu chýb v dátach. Vypočíta sa pred odoslaním správy a vloží do hlavičky:

Pole	Veľkosť	Popis
CRC16 Checksum	16 bitov	CRC16 kontrolný súčet

Po vložení týchto funkcionalít bude naša hlavička vyzeráť takto:



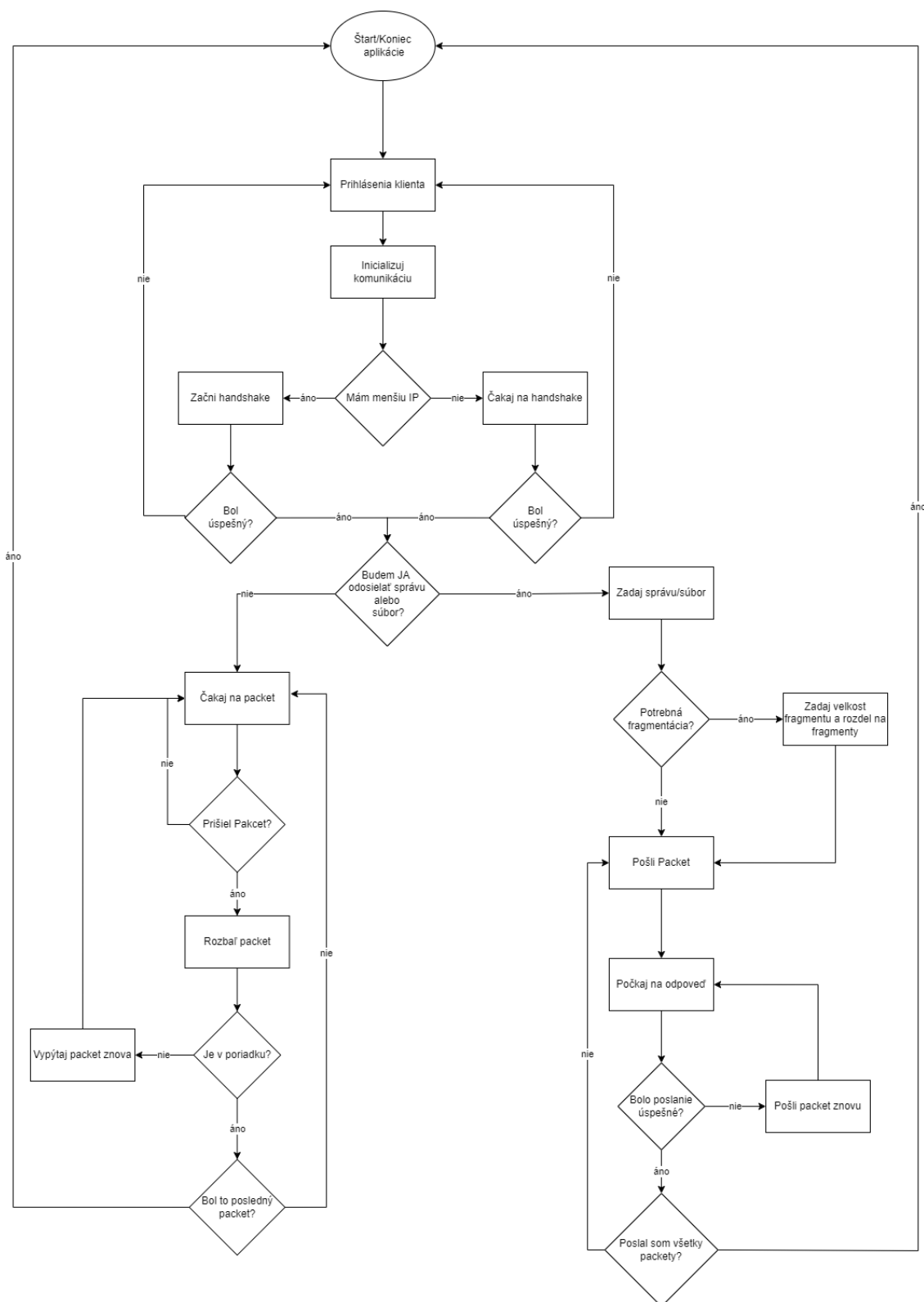
Takýmto spôsobom bude mať naša hlavička 12 Bytov plus veľkosť odosielaných dát.

Kombináciou týchto dát zabezpečíme vysokú kontrolu a správnosť odosielaných dát pri dostatočne vysokej rýchlosti prenosu.

Toto je plná **inicializačná hlavička**, keďže nie je potrebné neustále poskytovať niektoré polia ako napríklad celkový počet fragmentov.

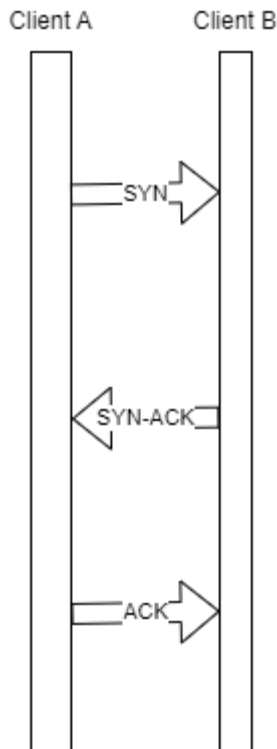
Diagram komunikácie

Tento diagram znázorňuje spôsob akým klienti postupujú pri komunikácii:



Funkcionality programu

Inicializácia komunikácie



Na zabezpečenie správneho spojenia najprv vykonáme Three-way handshake, čím sa uistíme, že máme vysokú synchronizáciu oboch strán.

Takýto handshake používa mnoho protokolov, keďže je vysoko spoľahlivý. Napr synchronizuje sekvencie oboch strán na prenos dát pri protokole TCP.

Funguje na takom princípe, že Klient A pošle najprv pošle počiatočný SYN packet, na ktorý reaguje Klient B kombinovaným SYN-ACK packetom. Potvrdí ním prijatie SYN packetu od Klienta A a pošle svoj vlastný synchronizačný packet. Na ten Klient A odpovie ACK packetom a spojenie je oficiálne.

No.	Time	Source	Destination	Protocol	Length	Info
115	3.986920	192.168.68.105	192.168.68.113	UDP	60	5503 → 5500 Len=2
116	3.987208	192.168.68.113	192.168.68.105	UDP	44	5500 → 5503 Len=2
117	3.990569	192.168.68.105	192.168.68.113	UDP	60	5503 → 5500 Len=2
185	8.953577	192.168.68.105	192.168.68.113	UDP	60	5503 → 5500 Len=6

Tento výpis bol testovaný pre kontrolný bod a teda hlavička má zatiaľ implementované iba 2B.

Ako vidíme z odchytenej komunikácie cez Wireshark, klient, ktorý inicializuje komunikáciu pošle SYN. Ak sa druhý klient nepripojí do 20 sekúnd, spojenie sa považuje za neúspešné a program sa ukončí. Naopak ak druhý klient 20 sekúnd neobdrží SYN, spojenie sa rovnako považuje za neúspešné a program sa ukončí. Posledný záznam sme poslali správu „ahoj“, ktorá má 4B wireshark nám ukazuje správne 6, z dôvodu, že sa posiela aj verzia a flagy z hlavičky.

ARQ metóda

Na zabezpečenie spoľahlivého prenosu dát nad nespoľahlivým protokolom UDP implementujeme metódu **ARQ (Automatic Repeat reQuest)**. Táto metóda umožňuje detekovať stratu dát a zabezpečuje opätovné odoslanie stratených alebo poškodených paketov.

Použijeme Stop&Wait metódu, kde prijímajúci klient bude čakať na fragment z danej správy a odpovie odosielateľovi ACK/NACK na základe prijatia správneho fragmentu. Keďže máme v hlavičke rozdelené sequence number a fragment number, budeme musieť obe tieto hodnoty kontrolovať, ale na druhú stranu vieme aj pri implementácii Stop&Wait metódy posilať viacej správ naraz.

Nevýhodou tejto metódy je skutočnosť, že prenos spomaľuje neustátnou systematickou kontrolou každého packetu, ale je zároveň najjednoduchší spôsob ako odhaliť poškodený/neodoslaný packet hneď.

CRC16 Checksum

V našej P2P aplikácii použijeme CRC16 checksum na zabezpečenie integrity prenášaných dát. Algoritmus využíva 16-bitový kontrolný súčet, ktorý dokáže odhaliť chyby pri prenose správ.

Na výpočet CRC16 začíname s hodnotou 0xFFFF a prechádzame každý byte dát. Najprv CRC XORujeme s aktuálnym bajtom, potom pre každý z 8 bitov kontrolujeme, či je najnižší bit CRC rovný 1. Ak áno, posunieme CRC doprava o 1 a XORujeme ho s polynómom 0xA001. Ak nie, len ho posunieme doprava.

Po výpočte pridáme výsledný CRC16 ako posledné 2 bajty správy. Na prijímacej strane opäť vypočítame CRC16 a porovnáme ho s prijatým kontrolným súčtom. Ak sa hodnoty zhodujú, správa je v poriadku; ak nie, uzol pošle požiadavku na opätovné odoslanie. Týmto spôsobom zabezpečujeme, že dáta sú prenášané bez chýb a zvyšujeme spoľahlivosť aplikácie.

Keep-Alive metóda

Implementujeme mechanizmus Keep-Alive, ktorý udržuje aktívne spojenie medzi uzlami a monitoruje ich aktivitu. Uzly posielajú "heartbeat" správy každých 5 sekúnd, aby potvrdili svoju prítomnosť. Ak jedna strana neobdrží odpoveď na tri po sebe idúce heartbeat správy, považuje spojenie za ukončené. V prípade výpadku spojenia sa uzol pokúsi overiť stav druhej strany a obnoviť prenos dát, ak je to možné.

Zmeny na základe inštrukcií cvičiaceho

Hlavička programu

Keďže hlavička navrhnutá v prvej verzii je príliš veľká a neefektívna z hľadiska významu UDP protokolu bolo nutné ju redukovať.

Na redukciu sme využili dva spôsoby. Prvý spôsob zabezpečuje, že pre režijné správy odosiela náš program iba 2 Bytovú hlavičku obsahujúcu verziu (identifikácia režijných správ) a flagy (ovládanie programu).

Pole	Veľkosť	Funkcia
Version	4 bity	Základná identifikácia packetu
Flags	12 bitov	Rôzne príkazy pre funkcie

Pri odosielaní dát využíva náš program plnú hlavičku, ktorá je ale redukovaná nasledovne:

Pole	Veľkosť	Funkcia
Version	4 bity	Základná identifikácia packetu
Flags	12 bitov	Rôzne príkazy pre funkcie
Sequence Number	16 bitov	Číslo sekvencie
Payload Length	16 bitov	Dĺžka dátovej časti
CRC	16 bitov	Celkový počet fragmentov

Odstránili sme nevyužívané pole **Acknowledgement number** a na číslovanie fragmentov sme nechali iba pole **Sequence number**, keďže náš program identifikuje posledný odosielaný fragment pomocou **FIN flag-u**.

Verzie a flagy sme upravili nasledovne:

Version	BIN	Dĺžka hlavičky	Funkcia hlavičky
FRG_V	0100	8 Bytov	Prenos dát
FLG_V	0010	2 Byty	Režijné správy pre kontrolu prenosu dát
HBT_V	1000	2 Byty	Režijné správy pre kontrolu Keep-alive
END_S	1110	2 Byty	Režijné správy pre ukončenie spojenia

Flag	Funkcia	Flag	Funkcia
SYN	00001	ACK	00010
FIN	00100	NAK	01000
FNM	10000		

Implementácia CRC v ARQ

Keďže sme v prvotnej dokumentácii mali popísané ako tieto metódy fungujú do finálnej verzie sme ich doimplementovali dokopy. Každý fragment, ktorý sa odošle potrebuje odozvu v podobe ACK, NAK alebo Timed out a zároveň potrebuje každý fragment kontrolu integrity. V našom programe, ak obdržíme fragment od odosielačujúceho klienta, najprv vykonáme výpočet CRC a na základe neho odpovedáme na prijatý fragment.

Konfigurácia programu

Do nášho programu sme doimplementovali funkciu na konfiguráciu do ktorej vie užívateľ vstúpi odoslaním správy „!“, ktorá sa neodošle, ale užívateľa presunie do konfiguračného módu. V tomto móde môže nastaviť maximálnu veľkosť fragmentu, nastaviť cestu kam bude ukladať prijatý súbor, simuláciu chybovosti a ukončenie spojenia.

Simulácia chybovosti

Aby sme vedeli otestovať všetky funkcionality nášho programu, doimplementovali sme aj simuláciu chybovosti. Tá je prirodzene vypnutá, ale dá sa nastaviť v konfiguračnom móde. Táto simulácia pozostáva z dvoch prvkov:

- **Percentuálna chybovosť** – percentuálna šanca na odoslanie zlej hodnoty CRC
- **Počet chybných fragmentov** – maximálny počet fragmentov, ktoré sa odošlú chybne

Wireshark

Našu komunikáciu sme odchytili aj cez wireshark a identifikovali jednotlivé polia. Vieme rozlíšiť typy správ, ktoré sa počas behu programu odosieli.

15	14.963700	127.0.0.1	127.0.0.1	APC	34	CONTROL
16	20.048945	127.0.0.1	127.0.0.1	APC	34	HEARTBEAT
17	20.051671	127.0.0.1	127.0.0.1	APC	34	HEARTBEAT
18	23.457448	127.0.0.1	127.0.0.1	APC	50	DATA -> 10 + 8 bytes
19	23.457673	127.0.0.1	127.0.0.1	APC	34	CONTROL
20	25.540953	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes
21	25.542906	127.0.0.1	127.0.0.1	APC	34	CONTROL
22	25.543050	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes
23	25.544898	127.0.0.1	127.0.0.1	APC	34	CONTROL
24	25.544999	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes
25	25.546835	127.0.0.1	127.0.0.1	APC	34	CONTROL
26	25.546953	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes
27	25.548960	127.0.0.1	127.0.0.1	APC	34	CONTROL
28	25.549089	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes
29	25.550933	127.0.0.1	127.0.0.1	APC	34	CONTROL
30	25.551029	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes
31	25.552891	127.0.0.1	127.0.0.1	APC	34	CONTROL
32	25.552977	127.0.0.1	127.0.0.1	APC	1064	DATA -> 1024 + 8 bytes